

Sesión 8: Despliegue de modelos (1) — El modelo como API con FastAPI

Duración: 2,5 horas | Modalidad: Virtual síncrona

Objetivo	Exponer el modelo de sentimientos como una API REST consumible por cualquier aplicación, en tiempo real.
Herramientas	FastAPI, Uvicorn, Pydantic, MLflow (carga de modelo), curl, Python requests
Prerrequisitos	Modelo SentimentAnalyzer en estado Production en el MLflow Registry (sesión 7).

BLOQUE 1 — Patrones de despliegue: Batch vs. Online (20 min)

Antes de escribir código, hay que elegir el patrón correcto para el caso de uso.

Batch (por lotes)	Online (tiempo real)
Cron job nocturno de scoring	API REST. Responde en < 200ms
Miles de registros de una vez	Una petición → una predicción
Ej: scoring de clientes para campaña	Ej: moderación en tiempo real de comentarios

En esta sesión construimos el patrón Online: un servicio web que recibe texto y devuelve una predicción de sentimiento de forma inmediata.

BLOQUE 2 — FastAPI: estructura básica (30 min)

Instalación:

```
pip install fastapi uvicorn[standard] pydantic mlflow transformers torch
```

FastAPI usa decoradores de Python para definir endpoints. Pydantic valida automáticamente los datos de entrada y genera documentación interactiva sin configuración extra.

Fichero: main.py — esqueleto inicial

```
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel, Field
from typing import List
import uvicorn

app = FastAPI(
    title='Sentiment Analysis API',
    description='Clasifica el sentimiento de textos usando DistilBERT.',
    version='1.0.0'
)

# — MODELOS DE DATOS (Pydantic) —————
class PredictRequest(BaseModel):
    text: str = Field(..., min_length=1, max_length=512,
                      example='This product is absolutely amazing!')

class PredictResponse(BaseModel):
    text: str
    label: str # 'POSITIVE' o 'NEGATIVE'
    score: float # Confianza del modelo (0.0 - 1.0)
    prediction: int # 1=positivo, 0=negativo

class BatchRequest(BaseModel):
    texts: List[str] = Field(..., min_items=1, max_items=32)

# — ENDPOINTS —————
@app.get('/health')
def health_check():
    '''Endpoint de salud: confirma que la API está activa.'''
    return {'status': 'ok', 'version': '1.0.0'}

@app.post('/predict', response_model=PredictResponse)
def predict(request: PredictRequest):
    '''Predicción de sentimiento para un solo texto.'''
    # Por ahora devolvemos un placeholder
    return {'text': request.text, 'label': 'POSITIVE', 'score': 0.99,
            'prediction': 1}

if __name__ == '__main__':
    uvicorn.run('main:app', host='0.0.0.0', port=8000, reload=True)
```

```
# Arrancar la API (modo desarrollo con recarga automática)
uvicorn main:app --reload --port 8000

# Abrir en el navegador:
# http://localhost:8000/docs → Swagger UI interactiva
# http://localhost:8000/redoc → Documentación alternativa
```

BLOQUE 3 — Integrar el modelo de MLflow (40 min)

Ahora conectamos el modelo que registramos en la sesión 7. La clave es cargar el modelo una sola vez al arrancar la API, no en cada petición.

Fichero: main.py — versión completa con MLflow

```
import os
import time
import logging
from contextlib import asynccontextmanager
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel, Field
from typing import List, Optional
import pandas as pd
import mlflow.pyfunc

# Configurar logging
logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)

# — CARGA DEL MODELO AL ARRANCAR —————
# Usamos lifespan para cargar el modelo una sola vez
model = None
model_info = {}

@asynccontextmanager
async def lifespan(app: FastAPI):
    global model, model_info
    model_name = os.getenv('MODEL_NAME', 'SentimentAnalyzer')
    model_stage = os.getenv('MODEL_STAGE', 'Production')
    mlflow_uri = os.getenv('MLFLOW_TRACKING_URI', 'http://localhost:5000')

    mlflow.set_tracking_uri(mlflow_uri)
    model_uri = f'models://{model_name}/{model_stage}'

    logger.info(f'Cargando modelo desde {model_uri}...')
    t0 = time.time()
    model = mlflow.pyfunc.load_model(model_uri)
    load_time = time.time() - t0

    model_info = {'name': model_name, 'stage': model_stage, 'load_time_s':
round(load_time, 2)}
    logger.info(f'Modelo cargado en {load_time:.2f}s')
    yield # La API está lista
    logger.info('Cerrando API...')

app = FastAPI(
    title='Sentiment Analysis API',
    description='API para clasificación de sentimientos en tiempo real.',
    version='1.0.0',
    lifespan=lifespan
)
```

```

# — MODELOS PYDANTIC —————
class PredictRequest(BaseModel):
    text: str = Field(..., min_length=1, max_length=512,
                      example='This product is absolutely amazing!')

class PredictResponse(BaseModel):
    text: str
    label: str
    score: float
    prediction: int
    latency_ms: float

class BatchRequest(BaseModel):
    texts: List[str] = Field(..., min_items=1, max_items=32)

class BatchResponse(BaseModel):
    results: List[PredictResponse]
    total_latency_ms: float

# — ENDPOINTS —————
@app.get('/health')
def health_check():
    return {
        'status': 'ok' if model is not None else 'loading',
        'model': model_info
    }

@app.post('/predict', response_model=PredictResponse)
def predict(request: PredictRequest):
    if model is None:
        raise HTTPException(status_code=503, detail='Modelo no disponible todavía')

    t0 = time.time()
    df = pd.DataFrame({'text': [request.text]})

    try:
        raw = model.predict(df)
    except Exception as e:
        logger.error(f'Error en predicción: {e}')
        raise HTTPException(status_code=500, detail=f'Error de predicción: {str(e)}')

    latency = (time.time() - t0) * 1000
    pred = int(raw[0])
    label = 'POSITIVE' if pred == 1 else 'NEGATIVE'

    logger.info(f'predict | text={request.text[:30]}... | label={label} | {latency:.1f}ms')

    return PredictResponse(
        text=request.text, label=label, score=0.95,
        prediction=pred, latency_ms=round(latency, 2)
    )

```

```

@app.post('/predict/batch', response_model=BatchResponse)
def predict_batch(request: BatchRequest):
    if model is None:
        raise HTTPException(status_code=503, detail='Modelo no disponible')

    t0 = time.time()
    df = pd.DataFrame({'text': request.texts})
    preds = model.predict(df)
    total_latency = (time.time() - t0) * 1000
    per_latency = total_latency / len(request.texts)

    results = []
    for text, pred in zip(request.texts, preds):
        p = int(pred)
        results.append(PredictResponse(
            text=text, label='POSITIVE' if p == 1 else 'NEGATIVE',
            score=0.95, prediction=p, latency_ms=round(per_latency, 2)
        ))

    return BatchResponse(results=results, total_latency_ms=round(total_latency,
2))

if __name__ == '__main__':
    import uvicorn
    uvicorn.run('main:app', host='0.0.0.0', port=8000, reload=False)

```

BLOQUE 4 — Probar la API de tres formas (20 min)

Forma 1: Swagger UI (interactiva, en el navegador)

Abrir <http://localhost:8000/docs> — permite probar los endpoints visualmente sin escribir código.

Forma 2: curl desde la terminal

```

# Health check
curl http://localhost:8000/health

# Predicción individual
curl -X POST http://localhost:8000/predict \
  -H 'Content-Type: application/json' \
  -d '{"text": "This movie was absolutely fantastic!"}'

# Predicción en batch
curl -X POST http://localhost:8000/predict/batch \
  -H 'Content-Type: application/json' \
  -d '{"texts": ["Great product!", "Terrible service.", "It was okay."']}'

# Test de error de validación Pydantic (texto vacío)
curl -X POST http://localhost:8000/predict \

```

```
-H 'Content-Type: application/json' \  
-d '{"text": ""}'
```

Forma 3: script Python con requests

```
# test_api.py  
import requests  
import json  
  
BASE_URL = 'http://localhost:8000'  
  
def test_health():  
    r = requests.get(f'{BASE_URL}/health')  
    print('Health:', r.json())  
    assert r.status_code == 200  
  
def test_predict():  
    cases = [  
        ('I love this product!', 'POSITIVE'),  
        ('Awful experience.', 'NEGATIVE'),  
    ]  
    for text, expected in cases:  
        r = requests.post(f'{BASE_URL}/predict', json={'text': text})  
        data = r.json()  
        status = 'OK' if data['label'] == expected else 'FAIL'  
        print(f'[{status}] "{text}" → {data["label"]}'  
              (f'{data["latency_ms"]:.1f}ms'))  
  
def test_batch():  
    texts = [  
        'Amazing!', 'Horrible.', 'Not bad.', 'Could be better.',  
        'Highly recommend!', 'Waste of money.'  
    ]  
    r = requests.post(f'{BASE_URL}/predict/batch', json={'texts': texts})  
    data = r.json()  
    print(f'Batch: {len(data["results"])} predicciones en'  
          {data["total_latency_ms"]:.1f}ms')  
    for res in data['results']:  
        print(f'  {res["label"]:8s} | {res["text"]}')  
  
if __name__ == '__main__':  
    test_health()  
    test_predict()  
    test_batch()
```

TRABAJO AUTÓNOMO (2,5 h)

Tarea para casa

Desarrollar la API completa con FastAPI para el modelo de sentimientos. Añadir un endpoint `/predict/explain` que devuelva también los tokens más influyentes en la predicción. Probar todos los endpoints con el script `test_api.py`. La semana que viene meteemos la API en un contenedor Docker.